

chatgpt給我寫的引導

files

Call tool

要求

{search_query: Array(3), scope: {...}, top_k: 8} 這是chatgpt調用的工具

https://chatgpt.com/s/t_6a9d0a3e16088191aec9f047366acab8

可以，而且我認為你的方向應該明確定成：

不要自己從零寫 Office。湛箴只自己做「工作流、Agent、人類主權、資料關係、UI 外殼」；Docs / Sheets 直接嵌入成熟開源引擎。

你目前的規格其實已經給出了非常好的邊界：Docs、Sheets、OCR、AI 路由、Hermes 都是同一個軟體中的能力，而四個面板是同一份資料循環的不同工作狀態。[我的代辦/GitHub/要真的幹活兒了/zhanzhen_workflow_and_visual_system_v1](#)

一、先說結論：我推薦這個組合

湛箴部分	建議	原因	對你的盈利模式
Docs	Tiptap / 或 BlockNote	成熟、React 友好、可自行做 Proton 風格 UI	✓
Sheets	Univer	真正接近 Excel/Google Sheets 的嵌入式開源方案	✓
XLSX 處理	SheetJS CE / ExcelJS	做匯入、匯出、資料解析	✓
即時協作	Yjs + Hocuspocus	本地/自架，不按人頭付費	✓
桌面版	Tauri 2	把 Web UI 包成 Windows/macOS/Linux	✓
圖譜	Cytoscape.js	關係圖、知識圖譜、依賴圖	✓
本地資料庫	SQLite	桌面端本地優先	✓
OCR	PaddleOCR	本機跑，不產生按量 API 成本	✓

你之前的總體架構其實已經選中了其中大部分：Tauri、Univer、Tiptap、Yjs、FastAPI、SQLite、PaddleOCR。[Ya-MiC-總體架構藍圖](#) [我的代辦/GitHub/要真的幹活兒](#)

[了/zhanzhen_workflow_and_visual_system_v1 Ya-MiC-一人公司系統憲法與業務白皮書-v1.0 我的代辦/GitHub/晏銘湛箴/zhanzhen_workflow_and_visual_system_v1 湛箴與ya-mic-os-主心骨 \(1\).](#)

二、你最關心的：Docs 到底有沒有可以直接拿來用的？

方案 A：Tiptap —— 我最推薦的長期方案

Tiptap 本身就是一個 headless rich-text editor，底層基於 ProseMirror，可以自己決定整個 UI。官方目前仍把 OSS Editor 定義為 MIT 授權。(Tiptap)

這對你非常重要。

因為你要的不是「一個像 Notion 的編輯器」。

你要的是：



Tiptap 的好處是：

它不強迫你接受它的視覺。

所以你完全可以自己把 Proton Docs 那種：

- 大留白
- 很弱的工具列
- 文件置中
- 柔和陰影

- 低對比邊框
- comments
- suggestion
- collaboration presence

做成你的「湛箴版本」。

而不是做成另一個 Tiptap。

三、BlockNote 其實也很適合你

BlockNote 更像：

「已經幫你做好一個 Notion 型 Docs。」

它現在有：

- block editor
- drag & drop
- nesting
- slash menu
- formatting menu
- realtime collaboration
- comments

而且官方目前說明大部分核心是 **MPL-2.0**，可以放進 commercial / closed-source application；但它的 xL 套件是 GPL-3.0，需要另外注意。(GitHub)

所以：

如果你的目標是「最快做出漂亮的 Docs」

BlockNote 更快。

如果你的目標是「湛箴最後要把 Docs 做成自己真正的核心」

Tiptap 更適合。

我的排序：

MVP：BlockNote



湛箴成熟版：Tiptap + ProseMirror

但有一個原則：

不要碰 BlockNote XL / GPL-3 套件。

四、Proton Docs 到底能不能直接拿？

這裡要非常小心。

Proton Drive 的 WebClients 現在確實是公開 GitHub，而且 Proton 也公開宣布 Proton Drive Apps 開源。(Proton) https://proton.me/support/drive-create-edit-docs?utm_source=chatgpt.com 但是 ProtonMail/WebClients 整個倉庫的 LICENSE 是：

GPL-3.0

([GitHub](#))

所以我不建議你：

```
Proton Docs source
    ↓
copy
    ↓
湛箴
    ↓
closed-source SaaS
```

這條路很容易把授權問題搞得非常麻煩。

但是：

```
Proton Docs
    ↓
研究 UI / interaction / information architecture
    ↓
自己使用 Tiptap
    ↓
自己重做湛箴 UI
```

完全是另外一回事。

而且 Proton Docs 自己的產品特徵，非常值得你借鑑：

- 文件左上角名稱
- 編輯器
- comments
- suggestions
- real-time collaboration
- presence
- version history
- secure sharing

Proton 官方現在就是這樣定位 Docs 的。(Proton)

所以你應該：

學 Proton Docs 的「設計思想」，不要拿 Proton Docs 的「產品程式碼」。

五、Sheets：我反而非常推薦 Univer

這個我基本可以替你定了：

湛箴 Sheets = Univer

Univer 現在就是專門做：

embedded spreadsheet / document / presentation experiences

的 framework，而且目前 repo 明確採 **Apache-2.0**。(GitHub)

這對你非常關鍵。

因為你的畫面不是：

湛箴
+
外掛 Excel

而應該是：

湛箴
|
└─ Docs

|
└ Sheets

同一個 Project 底下：

```
Project
├─ report.docx
├─ analysis.xlsx
├─ invoice-001.pdf
├─ meeting.md
├─ evidence.png
└─ decision.json
```

這和你自己的資料模型也是一致的。你的規格明確定義一個 Project 同時包含 Document 、 Spreadsheet 、 Asset 、 OCR Job 、 SOP 、 Meeting 、 Decision Record 、 Diff 等物件。 [我的代辦/GitHub/要真的幹活兒了/zhanzhen_workflow_and_visual_system_v1](#)

六、但是 Univer 有一個你現在必須知道的坑

這個非常重要。

Univer OSS 可以作為 Sheet 編輯器，但是 Office XLSX import/export 現在官方文件標示有服務端要求，而且部分完整 Office exchange 能力屬於 Pro 方向。 ([Univer AI](#))

也就是：

```
Excel 文件
  ↓
Univer
  ↓
編輯
```

沒有問題。

但：

```
任意 XLSX
  ↓
高保真匯入
  ↓
修改
  ↓
```

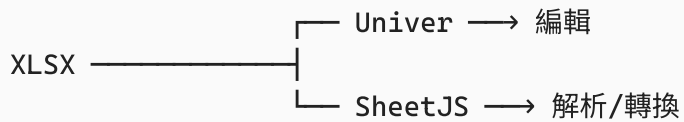
完全還原 Excel



高保真導出

這件事比「做一個 Sheet 編輯器」困難很多。

所以我建議你拆開：

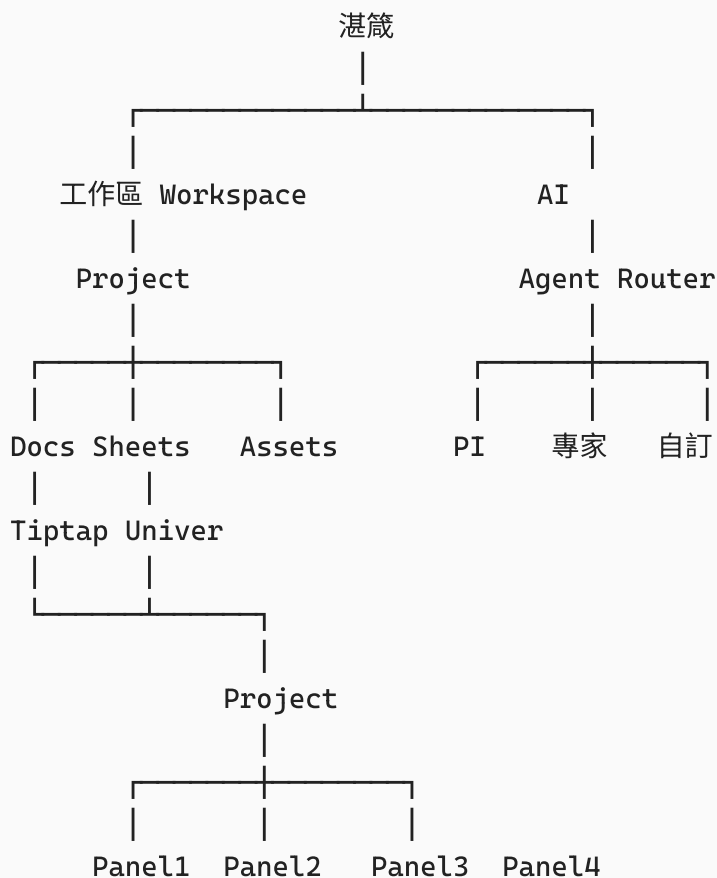


SheetJS Community Edition 目前仍是 Apache-2.0 開源，可以作為 Excel 資料解析/生成工具。
([GitHub](#))

ExcelJS 則是 MIT，也很適合用在 Node 端處理 `.xlsx`。([GitHub](#))

七、所以你真正應該做成這樣

我會把你這個「面板」重新分成：



這樣非常漂亮。

Docs 和 Sheets 不是兩個產品。

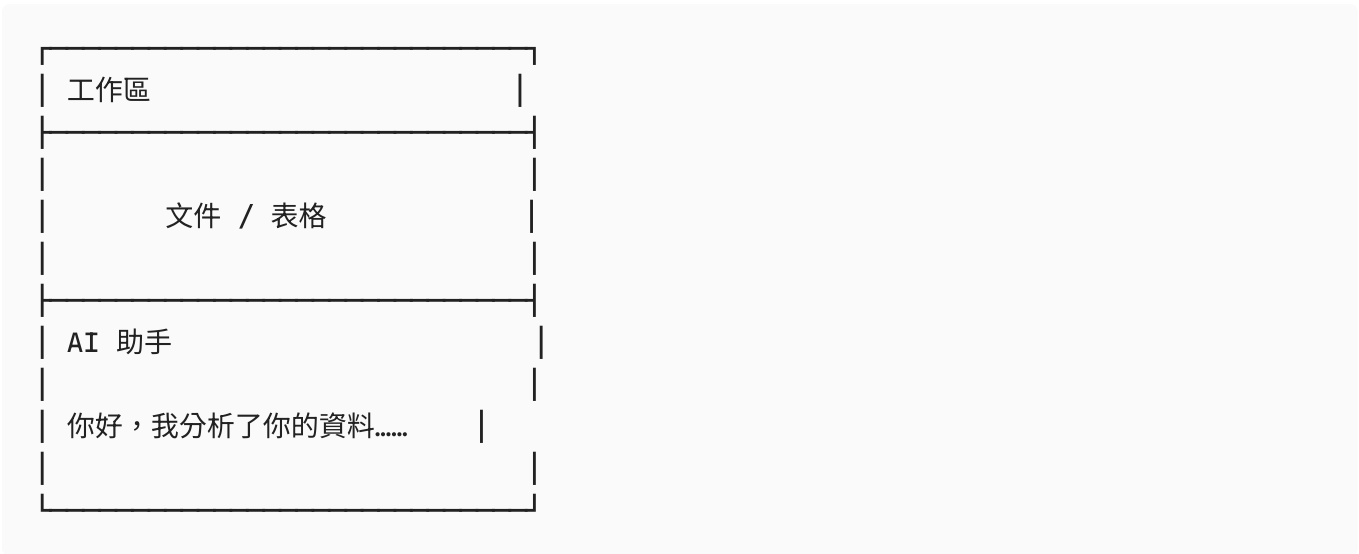
它們只是 Project 的兩種內容類型。

這正好符合你目前規格裡「湛箴是一個軟體」的定義。[我的代辦/GitHub/要真的幹活兒了/zhanzhen_workflow_and_visual_system_v1](#)

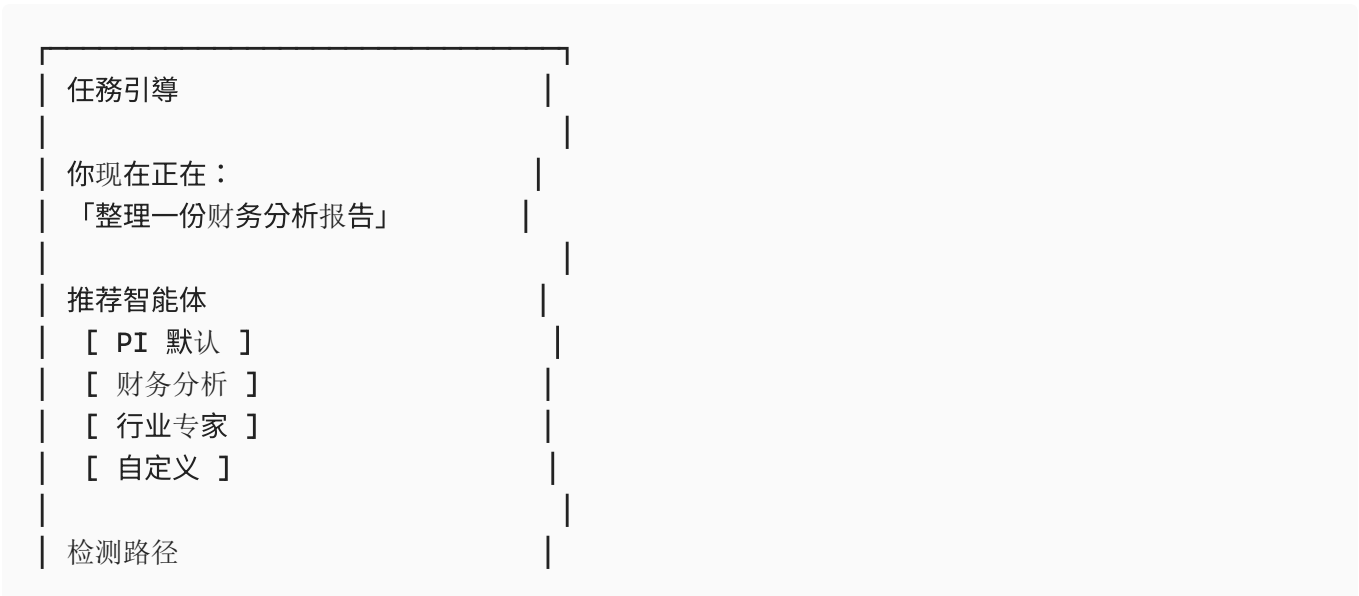
八、你右邊那個 AI 區域不要做成「ChatGPT」

這是你這個產品很容易做錯的地方。

你的圖片現在有：



我建議你改成：



- 本地文件
- 当前 Project
- 指定 Docs
- 指定 Sheets

AI 对话

.....

這就是你前面說的：

Agent 是 PI 架構，但引導程序要寫好；讓使用者可以選任何 Agent，預設 PI；並告訴使用者檢測路徑。

這其實是你的產品核心之一。

九、Agent 本身反而不用自己開發

你現在資金有限，我非常贊成你的策略：

開發期間

直接：

```
Cherry Studio
    +
Trae
    +
CodeBuddy
    +
GitHub
```

把它們當作：

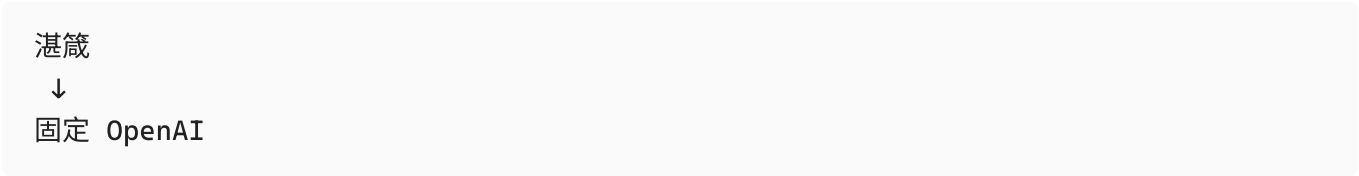
免費開發團隊

而不是拿它們當湛箴的 runtime。

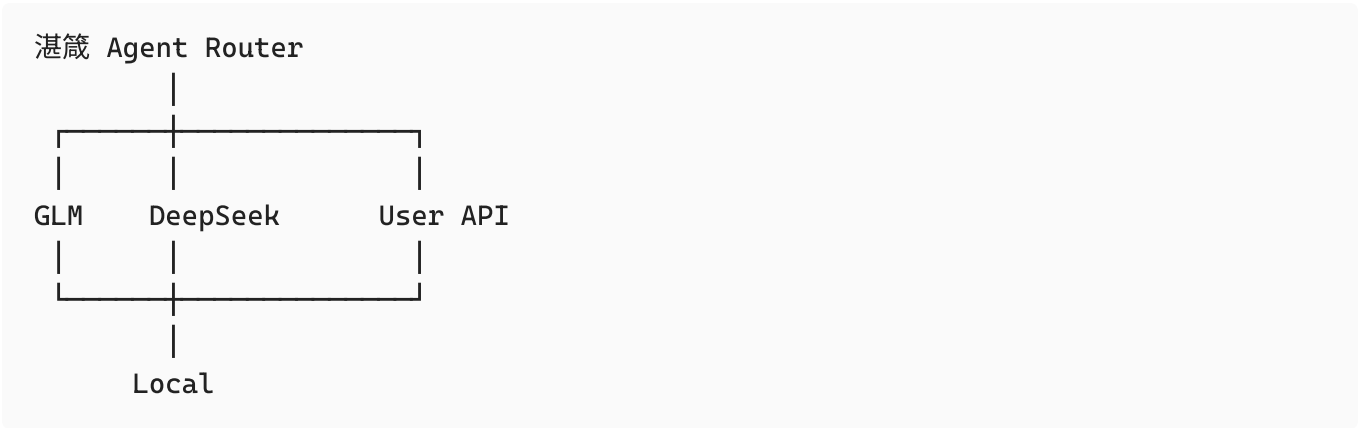
十、真正的湛箴 Runtime 要「可換」

這一點非常重要。

你的架構不要：

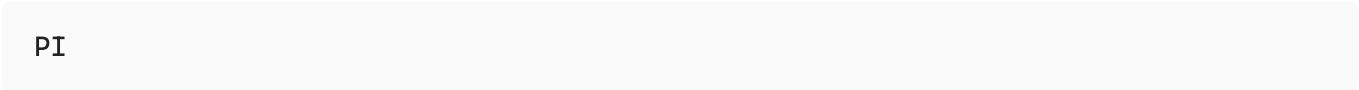


而是：



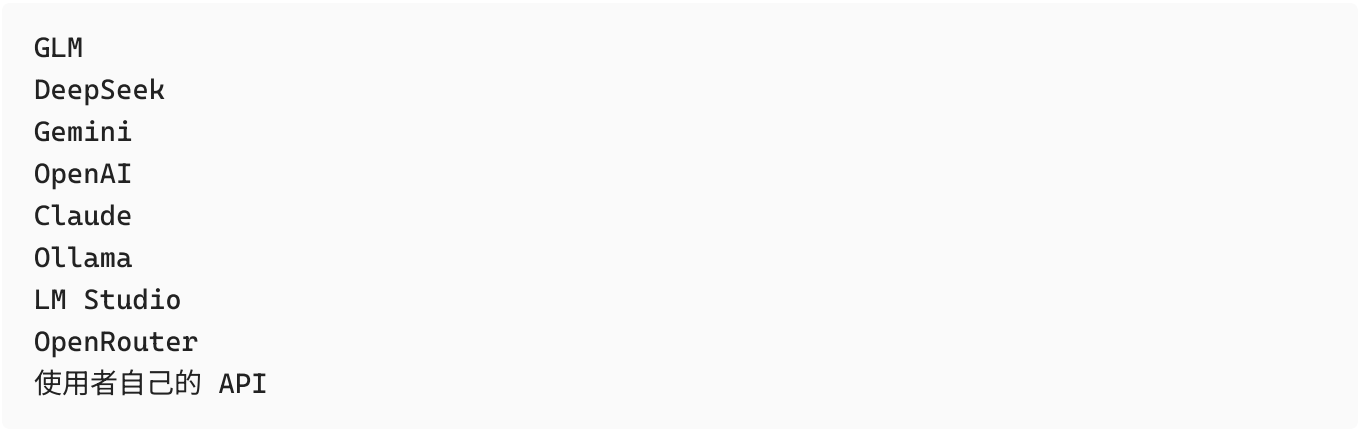
也就是：

預設



實際模型

可以是：



甚至：

```
PI
├─ Planner → GLM
├─ Analyst → DeepSeek
├─ Critic → Claude
└─ Localizer → Ollama
```

這才真正符合你的「智能體自由選擇」理念。

而你現在的規格本身也已經要求 AI provider 在發送前告知範圍與預估成本，且允許使用者自行切換 API。

[我的代辦/GitHub/要真的幹活兒了/zhanzhen_workflow_and_visual_system_v1](#)

十一、Tauri：非常適合你

你的最終桌面產品我依然建議：

```
React
  ↓
Tiptap / Univer
  ↓
Tauri 2
  ↓
Windows / macOS / Linux
```

Tauri 目前的程式碼採 MIT 或 MIT/Apache-2.0（依元件），定位就是把 Web frontend 包成桌面/行動應用。（[GitHub](#)）

所以你的：

「像 Cherry Studio 一樣，下載一個 exe，什麼都在裡面」

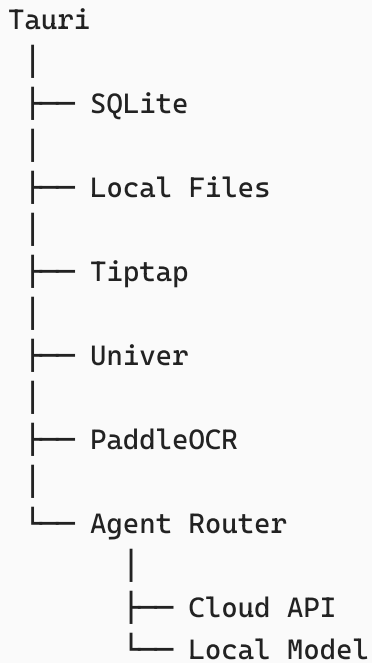
這條路是成立的。

十二、而且你可以做到「完全本地優先」

你以前規格裡最重要的一句其實是：

沒有網路、沒有 AI、沒有帳戶、沒有付費，打開即寫、寫完即存仍然成立。

所以桌面端可以：



這時候你的 **1.5 USDC/月會員** 才有意義。

因為：

會員費不是拿來支付每個人的 GPU。

而是支付：

帳戶
同步
雲端備份
工作區
協作
AI 路由
額度管理
高階功能

十三、Yjs + Hocuspocus 很適合第二階段

你要「張三和張三的 Agent 幹完活，現在兩個張三要不要見面」，其實已經開始碰 collaborative editing 了。

Yjs 目前是 MIT。([GitHub](#))

Hocuspocus 也是 MIT，而且就是給 Yjs 做 real-time collaboration backend。([GitHub](#))

所以：



以後甚至可以做到：

| Agent 在工作區裡留下自己的 revision / suggestion，真人選擇是否接受。

這跟你第四面板的 diff / 金標準 / 反哺非常吻合。

十四、Diff 千萬不要依賴 Tiptap 的付費產品

這裡我要特別提醒你。

Tiptap 基礎 Editor 是 MIT；但是目前官方的 **Tracked Changes** 是單獨的付費產品。([Tiptap](#))

所以你不要想：

```
Tiptap
+
買 Tracked Changes
```

這會開始侵蝕你 1.5 USDC 的 economics。

你自己的產品反而應該：



而你的 Diff 色彩規範正好已經定好了：

- + 淺薄荷
- 淡灰紫
- ↔ 藕粉

所以這個功能其實很適合你自己寫。



十五、ONLYOFFICE 我反而不建議你現在碰

它確實很強。

而且 Community Edition 免費。

但它的核心 DocumentServer 是 **AGPL-3.0**；企業版/Developer Edition則是 proprietary。
([GitHub](#))

這和你的商業模式：

湛箴
↓
閉源產品
↓
1.5 USDC / 月

不是最乾淨的組合。

所以：

ONLYOFFICE

功能：★★★★★

授權簡潔度：★★

你現在適配度：★★

十六、Luckysheet 也不要再選了

這個我可以直接替你排除。

Luckysheet 已經被官方宣布：

| no longer maintained

並且推薦直接使用 Univer；該 repo 於 2025 年 10 月已 archive。([GitHub](#))  [GitHub+1](#)

所以不要再讓 Agent 花時間研究：

Luckysheet

直接：

Univer

十七、GitHub 倉庫到底用什麼 License ？

這裡我要把「湛箴產品 License」和「GitHub 開源工具 License」分成兩件事。

A. 第三方

照原作者：

```
Tiptap → MIT
Univer → Apache-2.0
Yjs → MIT
Hocuspocus → MIT
Tauri → MIT / Apache-2.0
Cytoscape → MIT
SheetJS CE → Apache-2.0
ExcelJS → MIT
```

完全不要自己改別人的 License。

十八、湛箴自己的主程式

我的建議是：

不要一開始把湛箴完整產品開源。

你可以：

```
GitHub
|
├── zhanzhen
│   └── Private
|
├── zhanzhen-docs
│   └── Public
|
├── zhanzhen-schemas
│   └── Public
|
├── zhanzhen-skill
│   └── Public
|
```

└─ zhanzhen-demo
 └─ Public

核心：

湛箴主程式
→ All Rights Reserved

周邊：

Schema
Skill
SDK
Example
Documentation
Connector
→ Apache-2.0

這樣你就有：

Open Ecosystem + Closed Core

這個模式跟你的商業計畫比「整個 MIT」更合理。

你自己的規格目前其實已經寫成：

湛箴 v1 完整產品程式碼：閉源商業授權，All rights reserved。我的代辦/GitHub/要真的幹活
[兒了/zhanzhen_workflow_and_visual_system_v1](#)

我認為這個決定現在不用推翻。

十九、為什麼我不推薦你把主倉庫設 MIT？

因為一旦：

湛箴
MIT

別人可以：

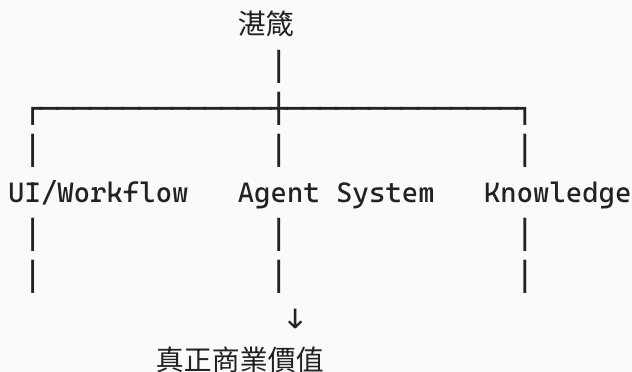
fork
↓
換品牌
↓
換模型
↓
自己部署
↓
收費

這對你的：

1.5 USDC / 月

並不是最有利。

你的護城河真正應該是：



而不是：

「我的 React 程式碼很神秘。」

二十、但你可以公開「湛箴 SDK / Plugin Protocol」

這個我非常建議。

例如：

zhanzhen-plugin-sdk

Apache-2.0

別人可以：

```
做一個財務 Agent
做一個律師 Agent
做一個 OCR Plugin
做一個 WPS Connector
做一個 Google Sheets Connector
```

然後接：

湛箴

這會比「把整個湛箴開源」更適合你。

二十一、你的最終工程結構，我會定成這樣

```
zhanzhen/
|
├─ apps/
|   ├── desktop/          ← Tauri
|   └─ web/
|
├─ packages/
|   ├── design-system/
|   ├── docs-editor/      ← Tiptap
|   ├── sheets-editor/    ← Univer
|   ├── diff-engine/
|   ├── agent-runtime/
|   ├── agent-router/
|   ├── project-model/
|   ├── knowledge-graph/
|   └─ plugin-sdk/
|
├─ services/
|   ├── sync/
|   ├── collaboration/    ← Hocuspocus
|   ├── ocr/
|   └─ api/
|
├─ local/
|   ├── sqlite/
|   └─ file-store/
```

```
|   └─ model/
|
|   └─ agents/
|       └─ pi/
|           └─ analyst/
|               └─ critic/
|                   └─ custom/
|
|   └─ docs/
|
|   └─ LICENSE
|   └─ NOTICE
|   └─ README.md
```

二十二、真正最重要的一點：不要一開始做完所有東西

你現在最大的資源不是錢。

是：

免費 Agent 的上下文與你的時間。

所以第一版只做：

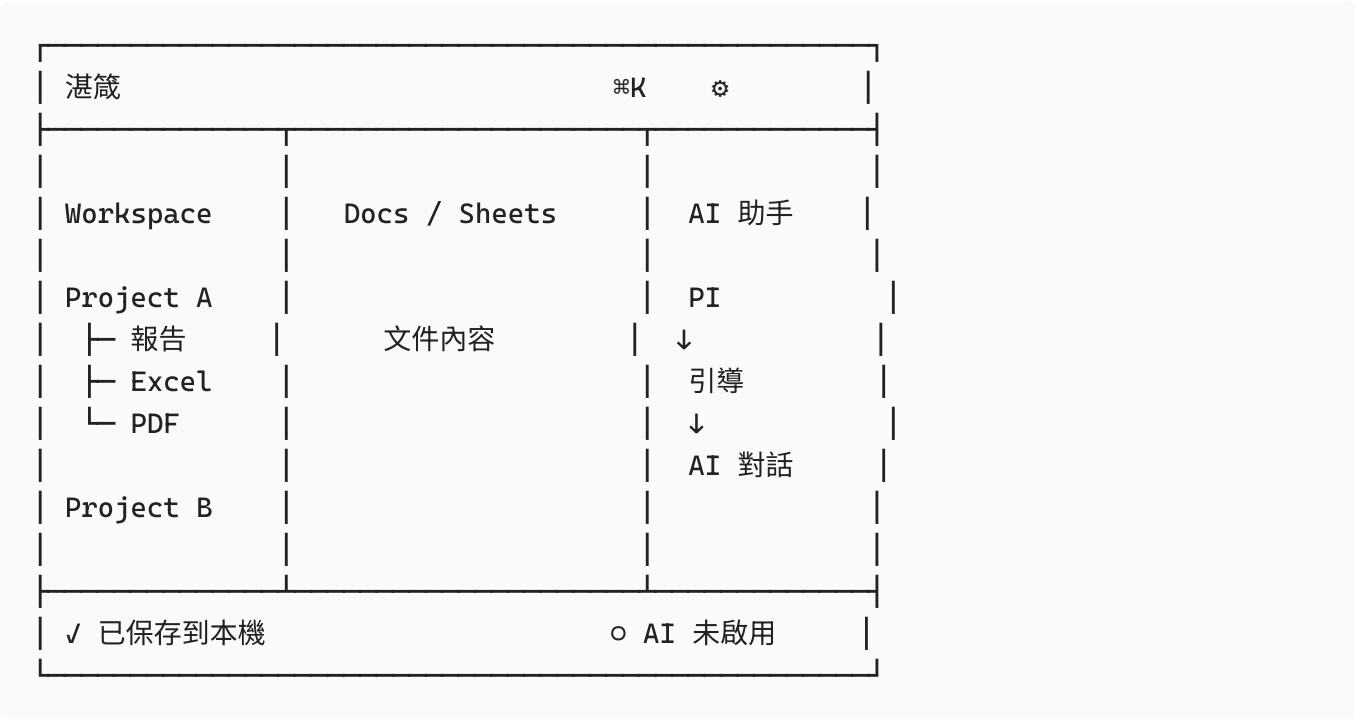
```
Tauri
+
React
+
湛箴 UI
+
Docs
+
Sheets
+
Project
+
左側工作區
+
右側 AI
```

然後讓：

① 人 + AI 共創

真正能跑起來。

第一個可用版本



這個東西其實已經是一個**真正的軟體**了。

再加：

- ② AI × AI
- ③ HUMAN ONLY
- ④ Diff / Golden Standard

你才慢慢形成完整的湛箴。

二十三、最後給你一個非常明確的「不花錢技術採購單」

類別	最終選擇
桌面	Tauri 2

類別	最終選擇
UI	React
Docs	Tiptap MIT
Docs 快速 MVP	BlockNote MPL-2.0
Sheets	Univer Apache-2.0
Excel 解析	SheetJS CE
XLSX Node 操作	ExcelJS
CRDT	Yjs
Collaboration	Hocuspocus
Graph	Cytoscape.js
Local DB	SQLite
OCR	PaddleOCR
Backend	FastAPI
AI Router	自己寫薄層
開發	Cherry Studio + Trae + CodeBuddy
Hosting	Cloudflare / 其他免費額度
核心產品 License	Closed / All Rights Reserved
公開 SDK / Schema / Skill	Apache-2.0

你的原有規格已經明確採用 Tauri、Tiptap、Univer、Yjs、SQLite、PaddleOCR 等路線，因此這不是推翻重做，而是把它從「技術清單」整理成一條真正能落地的低成本產品路線。[我的代辦/GitHub/要真的幹活兒了/zhanzhen_workflow_and_visual_system_v1](#)

我會直接替你做一個取捨

第一版：Tauri + React + Tiptap + Univer + SQLite。

不要 ONLYOFFICE。不要 Luckysheet。不要 Tiptap 付費 Tracked Changes。不要直接 fork Proton Docs。不要自己寫 Excel 編輯器。不要先做雲端多人協作。

先把你圖片裡的這個「工作區 + Docs/Sheets + 右側 Agent 引導」做真。

這樣你的 1.5 USDC/月不是拿來養一堆 SaaS 供應商，而是把錢留給真正有價值的同步、AI 用量與服務能力。

[Tiptap GitHub / OSS](#) [Univer GitHub](#) [BlockNote GitHub](#) [Yjs GitHub](#) [Hocuspocus GitHub](#) [Tauri GitHub](#)